

Kapitel 2 – Generics & Generische Binärsuche

Generische Klassen – Erklärung

Generische Klassen und Methoden erlauben es, Datentypen flexibel zu gestalten. Statt eine Klasse nur für `Integer` oder `String` zu schreiben, kann man mit Typparametern (``) denselben Code für unterschiedliche Typen wiederverwenden. Dies bringt *Wiederverwendbarkeit* und *Typsicherheit*.

Beispiel: Generische Klasse in Java

```
class Box<T> {
    private T inhalt;

    public void setInhalt(T wert) {
        this.inhalt = wert;
    }

    public T getInhalt() {
        return inhalt;
    }
}

public class GenericDemo {
    public static void main(String[] args) {
        Box<String> box1 = new Box<>();
        box1.setInhalt("Hallo");
        System.out.println(box1.getInhalt()); // Hallo

        Box<Integer> box2 = new Box<>();
        box2.setInhalt(123);
        System.out.println(box2.getInhalt()); // 123
    }
}
```

Generische Klasse im IHK-Pseudocode

```
klasse Box<T>
  private inhalt: T

  methode setInhalt(wert: T)
    inhalt = wert

  methode getInhalt() → T
    return inhalt
end klasse
```

Erklärung:

- `T` ist ein Platzhalter für den Typ, der beim Erzeugen der Klasse ersetzt wird. - Beispiel: `Box` → T = String, `Box` → T = Integer. - Vorteil: Der Compiler kann Typfehler verhindern, da kein Casten nötig ist.

Generische Binärsuche mit Key-Funktion

Die normale Binärsuche funktioniert nur mit Zahlen. Für Objekte (z. B. Kunde, Artikel, Bestellung) benötigt man eine *Key-Funktion*, die den Sortierschlüssel (z. B. Kundennummer) liefert.

Pseudocode:

```
funktion binaerSucheKey(L: Liste<T>, key: Funktion<T→Integer>, such: Integer)
    l = 0
    r = L.size() - 1
    solange l ≤ r
        m = (l + r) / 2
        k = key(L.get(m))
        wenn k == such dann return m
        sonst wenn k < such dann l = m + 1
        sonst r = m - 1
    end solange
    return -1
end funktion
```

Beispiel in Java:

```
import java.util.List;
import java.util.function.Function;

public class BinaerSucheKeyDemo {
    static <T> int binaerSucheKey(List<T> L, Function<T,Integer> key, int such) {
        int l = 0, r = L.size()-1;
        while (l <= r) {
            int m = l + (r - l) / 2;
            int k = key.apply(L.get(m));
            if (k == such) return m;
            else if (k < such) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }

    // Beispiel
    public static void main(String[] args) {
        record Kunde(String name, int nr) {}
        List<Kunde> kunden = List.of(new Kunde("Anna",10), new Kunde("Bernd",20), new Kunde("Carmen",30));

        int idx = binaerSucheKey(kunden, Kunde::nr, 20);
        System.out.println(idx); // 1
    }
}
```

Erklärung:

- `List` ist generisch und kann beliebige Objekte speichern. - `Function` übergibt eine Funktion, die aus T den Vergleichswert extrahiert (z. B. Kundennummer). - Vorteil: Diese Suche funktioniert für alle Objekttypen, solange ein int-Schlüssel definiert werden kann. - Typische IHK-Vorgaben: „Schreiben Sie eine generische Methode, die über $\text{key}(T) \rightarrow \text{int}$ vergleicht.“